Phil Ecosystem:
哲学研究のための計算基盤エコシステムの設計と実装

松井佑介 *

概要

哲学研究における計算的手法の導入は、言語学や歴史学と比較して著しく遅れている。その根本的原因は、哲学固有の課題 —— 概念の通約不可能性、解釈の多重性、テキストの多言語性（2500 年のスパンにわたるギリシャ語・漢文・サンスクリット語・現代諸語）、そして形式化に対する文化的抵抗 —— にある。本論文では、これらの課題に正面から向き合いつつ、哲学研究に計算的アプローチを「日常化」するためのエコシステム Phil Ecosystem の設計と実装を報告する。

Phil Ecosystem は、生物情報学における Bioconductor の成功パターンに学び、7 層アーキテクチャ（計算バックエンド、リポジトリ、アノテーション、定量化エンジン、コアデータ構造、専門分析、統合ワークベンチ）を採用する。中核となるデータコンテナ PhilCorpus は、Bioconductor の SummarizedExperiment を哲学データに適応させたものであり、テキスト・埋め込み・特徴量の多層構造に加え、哲学固有の「解釈の多重性」次元を第一級オブジェクトとして組み込む。研究者は約 15 の「動詞」（`phil_read`, `phil_explore`, `phil_compare`, `phil_plot` 等）のみで全機能にアクセスでき、内部の 7 層構造を意識する必要がない。

R/Python 二言語戦略により、デジタル・ヒューマニティーズ研究者と機械学習研究者の双方に自然なインターフェースを提供する。哲学ドメイン特化のファインチューニング済み埋め込みモデル（philmap-e5-v2, Spearman $\rho = 0.976$）により、和辻の「間柄」からハイデガーの Mitsein、ブーバーの Ich-Du、ウブントゥ哲学まで、言語・伝統を超えた概念検索を実現する。本エコシステムは、「計算が哲学に対して、望遠鏡が天文学に対して果たした役割を果たす」というビジョンのもと、個人の認知限界を超えた哲学研究の新たなパラダイムを構築することを目指す。

キーワード: デジタル・ヒューマニティーズ, 計算哲学, 比較哲学, 概念整列, 多言語埋め込み, 知識グラフ, ソフトウェアエコシステム

目次

1 はじめに 5
- 1.1 哲学研究における情報技術導入の現状 5
- 1.2 他の人文学との比較：なぜ哲学は遅れているのか 5
- 1.3 本エコシステムのビジョン 5
- 1.4 論文の構成 6

2 哲学研究に計算的アプローチが必要な理由 6
- 2.1 単一学者の認知限界問題 6
- 2.2 概念の通約不可能性問題 7
- 2.3 解釈の追跡可能性問題 7

* Email: matsui@example.ac.jp

2.4 正典中心主義からの脱却 8

3 日常化のための設計思想 8
3.1 Bioconductor の成功に学ぶ 8
3.2 「BLAST のような概念検索」 9
3.3 研究者が触れるのは 15 の「動詞」だけ 10

4 具体的利用事例 10
4.1 事例 1：和辻「間柄」の伝統横断探索 11
4.2 事例 2：概念比較レポートの自動生成 12
4.3 事例 3：概念の時系列変遷追跡 13
4.4 事例 4：解釈の布置図 14
4.5 事例 5：ゼミ・教育利用 15
4.6 事例 6：科研費申請・論文準備ワークフロー 16

5 データ構造：PhilCorpus と ConceptSpans 16
5.1 PhilCorpus：哲学テキストのための標準データコンテナ 16
5.2 ConceptSpans：テキスト上の概念アノテーション 19
5.3 解釈の多重性への対応 19

6 エコシステムアーキテクチャ 20
6.1 7 層アーキテクチャ 20
6.2 R/Python 二言語戦略 21
6.3 リポジトリ構成 22
6.4 依存関係グラフ 22

7 テキスト定量化エンジン（philengine） 23
7.1 プラガブルバックエンド設計 23
7.2 ファセット埋め込み 24
7.3 非ベクトル的定量化 26
7.4 古典語への対応 26
7.5 哲学ドメイン特化ファインチューニング 27

8 概念整列手法（philmap） 27
8.1 4 つの整列手法 27
8.2 ベンチマーク 29

9 知識グラフとテキスト解析 30
9.1 philgraph：哲学知識グラフ 30
9.2 philtext：論証抽出と解釈追跡 31
9.3 非古典論理サポート 32

10 深層学習基盤との関係 32

10.1 大規模言語モデルとの連携 32
10.2 多言語埋め込みモデルの発展と哲学への応用 33
10.3 今後のモデル開発の方向性 33

11 関連研究 34
11.1 デジタル・ヒューマニティーズ 34
11.2 計算哲学 34
11.3 哲学的テキストマイニング 34
11.4 Bioconductor / BioPython との比較 35
11.5 多言語埋め込みモデル 35

12 ロードマップと今後の課題 35
12.1 実装ロードマップ 35
12.2 コミュニティ形成の課題 36
12.3 評価基準の共同構築 36

13 まとめ 36
13.1 本質的な貢献 36
13.2 「哲学のパラダイムシフト」への展望 37

1　はじめに

1.1　哲学研究における情報技術導入の現状

21 世紀に入り、人文学の諸分野は情報技術との融合を急速に進めてきた。言語学では大規模コーパスに基づく統計的手法が標準的方法論となり、歴史学では GIS（地理情報システム）やテキストマイニングによるデジタル・アーカイブの構築が進んでいる。考古学ではフォトグラメトリやライダー（LiDAR）による三次元記録が不可欠な技術となった。文学研究においても、Franco Moretti の「遠読（distant reading）」[18] 以降、計算的テキスト分析は広く受容されつつある。
しかし、哲学研究は他の人文学分野と比較して、計算的手法の導入が顕著に遅れている。PhilPapers[16] や InPhO（Indiana Philosophy Ontology）[17] のような知識基盤は存在するものの、これらは哲学研究の「道具」というよりも「資料索引」にとどまっている。哲学者の日常的な研究 —— テキストの精読、概念の比較、論証の分析 —— に計算的手法が統合されている事例はきわめて少ない。

1.2　他の人文学との比較：なぜ哲学は遅れているのか

この遅れには構造的な理由がある。第一に、哲学が扱うデータは本質的に「解釈を要する」ものであり、テキストの統計的処理が直ちに哲学的洞察につながるわけではない。語の出現頻度を数えても、カントの超越論的演繹の構造は見えてこない。この点で、哲学は言語学の統計的アプローチとは異なる方法論を必要とする。
第二に、哲学テキストの言語的多様性は他分野と比較しても際立っている。古典ギリシャ語、ラテン語、サンスクリット語、漢文、アラビア語、そして現代の英語・ドイツ語・フランス語・日本語など、2500 年以上のスパンにわたる多言語テキストを横断的に扱う必要がある。歴史学も多言語テキストを扱うが、特定の時代・地域に限定されることが多い。哲学では、紀元前 5 世紀のギリシャのテキストと 20 世紀の日本のテキストを同じ概念的枠組みで比較するという要求が日常的に生じる。
第三に、そしておそらく最も本質的な理由として、哲学者コミュニティには形式化に対する根深い懐疑がある。概念の計算的処理は、概念の豊かさの「矮小化」と見なされがちである。ハイデガーの Dasein をベクトルに変換することに、現象学者が抵抗を感じるのは自然なことかもしれない。しかし、これは望遠鏡の導入に対して「肉眼で見えないものは天体ではない」と主張するようなものでもある。

1.3　本エコシステムのビジョン

本論文で提案する Phil Ecosystem は、次のビジョンに基づいている：

計算は哲学に対して、望遠鏡が天文学に対して果たしたのと同じ役割を果たせるか？

望遠鏡は「肉眼では見えなかったもの」を見えるようにし、天文学のパラダイムを変えた。ガリレオが木星の衛星を発見したとき、それは望遠鏡なしでは原理的に不可能な発見であった。同様に、Phil Ecosystem は「いかなる個人の読解でも到達できなかった概念間の関係」を計算的に可視化することを目指す。
例えば、和辻哲郎の「間柄」概念と南アフリカのウブントゥ哲学における「umuntu ngumuntu

ngabantu（人は他の人々を通じてこそ人である）」という命題の間に、構造的な類似性はあるだろうか。この問いに答えるには、日本語とズールー語の両方を読める哲学者が必要であるが、そのような研究者はほぼ存在しない。計算的アプローチは、この「個人の言語能力の限界」という構造的制約を超える可能性を持つ。もちろん、計算が「発見」するのは類似性の候補であり、その哲学的意義の評価は依然として人間の仕事である。しかし、候補の発見すらなければ、評価も始まらない。

1.4　論文の構成

本論文は、異なる読者層を意識して前半と後半に分かれている。
前半（第 2 章 – 第 6 章）は主に哲学研究者を対象とし、計算的アプローチの必要性（第 2 章）、日常化のための設計思想（第 3 章）、具体的な利用事例（第 4 章）、データ構造の概要（第 5 章）を平易に説明する。哲学研究者がこの部分だけを読んでも、Phil Ecosystem が自身の研究にどう役立つかを理解できることを意図している。
後半（第 6 章 – 第 10 章）はインフォマティクス研究者を対象とし、7 層アーキテクチャの詳細（第 6 章）、テキスト定量化エンジン（第 7 章）、概念整列手法（第 8 章）、知識グラフとテキスト解析（第 9 章）、深層学習基盤との関係（第 10 章）を技術的に記述する。
第 11 章で関連研究を概観し、第 12 章でロードマップを示し、第 13 章でまとめる。

2　哲学研究に計算的アプローチが必要な理由

2.1　単一学者の認知限界問題

現在の比較哲学研究は、個々の研究者の言語能力と読書量に根本的に制約されている。ある研究者がドイツ語と日本語を読めるとき、ハイデガーと西田幾多郎の比較は可能になる。しかし、その同じ研究者がズールー語やサンスクリット語を読めないとき、ハイデガーとウブントゥ哲学、あるいは龍樹の中観思想との比較は不可能である。これは個人の努力の問題ではなく、構造的な限界である。
この構造的限界がもたらす帰結は深刻である。比較哲学の研究は、少数の言語的交差点（英独、英仏、英中など）に集中し、それ以外の比較可能性は体系的に見落とされる。
具体例を挙げよう。和辻哲郎の「間柄」概念は、人間存在の本質を他者との関係性において捉える。この概念と構造的に類似し得る概念は、世界の哲学的伝統の中に多数存在する可能性がある：

- ハイデガーの Mitsein（共存在）—— 存在論的に他者との共にあることを主張
- ブーバーの Ich-Du（我と汝）—— 対話的関係の根源性を主張
- ウブントゥ哲学の「umuntu ngumuntu ngabantu」—— 人は他者を通じて人となる
- 儒教の「仁」—— 他者への仁愛としての人間本質
- 仏教の「縁起」（pratītyasamutpāda）—— 関係性の存在論的根源性
- レヴィナスの他者論 —— 他者の顔による倫理的呼びかけ

しかし、これらすべてを原典で読み比べることができる研究者は存在しない。日本語・ドイツ語・ヘブライ語・ズールー語・古典中国語・サンスクリット語・フランス語をすべて読める哲学者を想定することは、現実的ではない。
計算的アプローチは、この限界を二つの方法で超克する。第一に、多言語埋め込みモデルにより、異

なる言語のテキストを共通の意味空間に射影し、言語横断的な類似性検索を可能にする。第二に、知識グラフにより、概念間の構造的関係（上位概念、対立概念、影響関係など）を言語に依存しない形で記述し、グラフ理論的な類似性分析を可能にする。

2.2 概念の通約不可能性問題

異なる哲学的伝統に属する概念を「比較する」とは、そもそも何を意味するのだろうか。和辻の「間柄」とハイデガーの Mitsein が「類似している」と言うとき、それはどのレベルでの類似を指すのか。意味的（semantic）に類似しているのか、構造的（structural）に —— すなわちそれぞれの体系内で類似した位置を占めているのか、機能的（functional）に —— すなわち論証において類似した役割を果たすのか。

この問題は、概念の通約不可能性（incommensurability）として知られる [25]。クーンが科学理論のパラダイム間で指摘したこの問題は、哲学的伝統間ではさらに先鋭化する。なぜなら、哲学的概念はそれ自体が理論的文脈に深く埋め込まれており、文脈から切り離して比較すること自体が概念の変質を招く可能性があるからである。

Phil Ecosystem はこの問題に対して、独断的な解決を提示するのではなく、通約不可能性の程度を多次元的に可視化するという戦略をとる。具体的には、4 つの整列手法 —— 意味的整列（embedding cosine similarity）、構造的整列（グラフベースの位置比較）、論証的整列（論証における役割の比較）、ハイブリッド整列（3 手法の重み付き結合） —— を提供し、研究者が各次元での類似度を個別に評価できるようにする（第 8 章で詳述）。

さらに重要な点として、形式化の政治性という問題がある。西洋哲学で発展したカテゴリ体系（存在論、認識論、倫理学、美学など）で非西洋哲学を記述することは、それ自体が一つの理論的選択であり、場合によっては認識的暴力（epistemic violence）となりうる。例えば、仏教の「空」（śūnyatā）を「存在論的カテゴリ」として分類すること自体が、「空」の意味を歪める可能性がある —— なぜなら、龍樹において「空」はまさに「あらゆるカテゴリの否定」を意味するからである [4]。

Phil Ecosystem のデータモデルは、この問題に対して、非古典論理の第一級サポートを提供する（第 5 章 5.3 節参照）。四句分別（catuskoti）、場所論理（basho logic）、矛盾許容論理、弁証法論理を、西洋の古典論理と同格のフレームワークとして実装する。

2.3 解釈の追跡可能性問題

哲学テキストの解釈は、自然科学のデータとは本質的に異なる性質を持つ。同一のテキストに対して、複数の正当な解釈が並立しうる。ハイデガーの『存在と時間』[2] 第 26 節における Mitsein の解釈を例にとろう。

ドレイファスはハイデガーをプラグマティスト的に読み、Mitsein を日常的な道具使用における他者との共同性として解釈する [8]。リチャードソンはより現象学的に読み、Mitsein を存在了解の共有構造として解釈する [9]。レヴィナスはハイデガーを批判的に読み、Mitsein が他者の倫理的次元を十分に捉えていないと論じる [7]。

これら三つの解釈はいずれも学術的に正当であり、いずれか一つだけが「正しい」とは言えない。しかし現在の哲学研究の実践では、これらの解釈の分岐と変遷は体系的に記録されていない。どの学者がどの時点でどの解釈を採用し、それがどのように変化してきたかを追跡することは、個別の研究論

文を丹念に追わない限り困難である。
Phil Ecosystem は、この解釈の追跡可能性（hermeneutic traceability）をデータ構造のレベルで保証する。PhilCorpus データコンテナは、同一テキストに対する複数の解釈を「解釈レイヤー」として第一級に扱い、各解釈に注釈者・学派・年代のメタデータを付与する（第 5 章で詳述）。

2.4 正典中心主義からの脱却

西洋哲学の研究は、プラトン、アリストテレス、カント、ヘーゲル、ハイデガーなど少数の「正典（canon）」テキストに集中する傾向がある。東洋哲学の研究も同様に、『論語』『道徳経』『中論』など少数の正典に集中しがちである。この正典中心主義は、周縁化された哲学的伝統 —— アフリカ哲学、ラテンアメリカ哲学、先住民族の哲学的伝統など —— の体系的な不可視化をもたらしている。
計算的アプローチは、この問題に対して、コーパスの規模を劇的に拡大する可能性を提供する。人間の精読には限界があるが、計算的処理は大量のテキストを対象とすることができる。Phil Ecosystem の PhilCorpusHub（第 6 章参照）は、正典テキストだけでなく、周縁化されたテキスト・伝統を含む多様なコーパスを提供することを目指している。
もちろん、計算的処理は精読の代替ではない。しかし、「どのテキストを精読すべきか」の候補を既存の正典を超えて発見することは、計算的アプローチの重要な貢献となりうる。和辻の「間柄」を探索した結果として、これまで比較哲学の対象とされていなかったウブントゥ哲学のテキストが候補に挙がるならば、それは正典中心主義を超える一歩となる。

3 日常化のための設計思想

計算的手法が哲学研究に定着するためには、ツールが研究者の「日常」に溶け込む必要がある。本章では、Phil Ecosystem の設計思想を、生物情報学における Bioconductor の成功事例を参照しながら説明する。

3.1 Bioconductor の成功に学ぶ

Bioconductor[10] は、ゲノム解析のための R パッケージ・エコシステムであり、2002 年の創設以来、生物情報学のデファクト・スタンダードとなっている。その成功の本質は、DESeq2 や edgeR のような解析ツールにあるのではなく、SummarizedExperiment[11] という共通データコンテナにある。
SummarizedExperiment は、遺伝子発現データを「行（遺伝子）× 列（サンプル）× 層（アッセイ）」の三次元構造として標準化した。この標準化により、異なる研究グループが生成したデータをシームレスに組み合わせることが可能になり、数百の解析パッケージが SummarizedExperiment を入出力として相互運用できるようになった。
もう一つの重要な要素は GRanges（genomic ranges）である。GRanges は「ゲノム上の位置」を表す標準的なデータ構造であり、染色体名、開始位置、終了位置、ストランドの情報を持つ。この共通表現により、遺伝子アノテーション、変異情報、エピゲノムデータなど異種のゲノムデータを位置情報を介して統合することが可能になった。
Phil Ecosystem はこの成功パターンを哲学データに適応する。表 1 に、Bioconductor と Phil

Ecosystem の対応関係を示す。

表 1　Bioconductor と Phil Ecosystem の構造的対応

Bioconductor	**Phil Ecosystem**	**役割**
SummarizedExperiment	PhilCorpus	コアデータコンテナ
GRanges / IRanges	ConceptSpans	テキスト上の概念位置
MultiAssayExperiment	PhilCollection	複数コーパスの統合
org.Hs.eg.db	phil.concepts.db	概念アノテーション DB
TxDb.*	phil.traditions.db	伝統構造 DB
BSgenome.*	phil.texts.*	テキストパッケージ
ExperimentHub	PhilCorpusHub	キュレーション済みコーパス
AnnotationHub	PhilAnnotationHub	アノテーションリソース
Rsubread / STAR	philengine	生データ → 定量値変換
DESeq2	philmap	差次的解析（概念比較）
iSEE	phil_dashboard()	インタラクティブ可視化
plyranges	phil_explore()	tidy 的データ探索
ComplexHeatmap	phil_concept_map()	高度な可視化
clusterProfiler	phil_compare()	エンリッチメント分析

SummarizedExperiment がなぜゲノム研究を変えたかを理解することは重要である。それ以前、研究者は遺伝子発現データをアドホックな形式で管理しており、異なるツール間でのデータ変換が常に必要であった。SummarizedExperiment は「データの持ち方」を標準化することで、ツール間の摩擦を劇的に低減した。PhilCorpus は哲学データにおいて同じ役割を果たすことを目指す。

3.2　「BLAST のような概念検索」

分子生物学において、BLAST（Basic Local Alignment Search Tool）[12] は研究のパラダイムを変えた検索ツールである。研究者は新たに発見した遺伝子配列を入力として、世界中のデータベースから類似配列を瞬時に検索できる。化学においては SciFinder[27] が同様の役割を果たし、化合物の構造式を入力として関連する化学物質や反応を検索できる。
Phil Ecosystem が目指す「概念的 BLAST」は、哲学テキストの一節を入力として、言語・伝統を超えて概念的に親縁性のあるパッセージを検索する機能である。

研究者が和辻の『倫理学』から「人間の存在は本質的に他者との間柄においてある」という一節を入力すると、ハイデガーの Mitsein に関する記述（ドイツ語）、ブーバーの Ich-Du に関する記述（ヘブライ語/ドイツ語）、ウブントゥ哲学の関連テキスト（英語/ズールー語）、儒教の「仁」に関する記述（古典中国語）が、類似度スコアとともに返される。

この機能は、BLAST が既知のタンパク質ドメインの発見を加速したように、既知の哲学的概念の伝統横断的な類縁関係の発見を加速する。重要なのは、検索結果は「仮説の候補」であり、哲学的吟味を代替するものではないという点である。BLAST のヒットも実験的検証を必要とするのと同様に、概念的 BLAST のヒットも哲学的検証を必要とする。

3.3　研究者が触れるのは 15 の「動詞」だけ

Phil Ecosystem の全体は 7 層・13 以上のパッケージで構成されるが（第 6 章で詳述）、哲学研究者が直接触れるのは統合ワークベンチ層（philworkbench / philbench）の約 15 の「動詞」だけである。これはファサード設計パターン [24] の応用であり、複雑な内部構造を隠蔽して直感的なインターフェースを提供する。
表 2 に、主要な動詞とその機能、内部で呼び出される層を示す。

表 2　Phil Ecosystem の動詞体系

カテゴリ	動詞	機能	内部層
読む	`phil_read()`	テキスト読み込み	Hub, Core
	`phil_lookup()`	概念・思想家検索	DB
定量化	`phil_embed()`	テキスト埋め込み	Engine
	`phil_quantify()`	特徴量抽出	Engine
探索	`phil_explore()`	伝統横断探索	Engine, Map
	`phil_search()`	類似概念検索	Map
比較	`phil_compare()`	概念比較	Map
	`phil_align()`	概念整列	Map
	`phil_trace()`	影響・系譜追跡	Text
注釈	`phil_annotate()`	自動注釈	Text
	`phil_review()`	人間レビュー	Workbench
可視化	`phil_plot()`	汎用可視化	Workbench
	`phil_dashboard()`	Shiny UI	Workbench
出力	`phil_to_latex()`	LaTeX 出力	Workbench
	`phil_export()`	JSON-LD/RDF	Core

この設計により、哲学研究者は「テキストを読み込んで、間柄の類似概念を探索し、結果をプロットする」という自然な研究フローを、3 行のコードで表現できる：

```
corpus <- phil_read("watsuji_rinrigaku")
results <- phil_explore("間柄", traditions = "all", top_k = 30)
phil_plot(results, type = "tradition_map")
```

Listing 1　Phil Ecosystem の基本的なワークフロー

内部では、`phil_explore()` が phil.concepts.db で概念 ID を解決し、PhilCorpusHub から関連テキストを取得し、philengine で埋め込みを計算し（キャッシュがあればスキップ）、philmap で整列を実行して結果を返す —— しかし研究者はこれらの層を意識する必要がない。

4　具体的利用事例

本章では、Phil Ecosystem の具体的な利用事例を 6 つ提示する。各事例は実際のコードと期待される出力を含み、哲学研究者がシステムの有用性を具体的に評価できることを目指す。

4.1　事例 1：和辻「間柄」の伝統横断探索

和辻哲郎の『倫理学』[1] における「間柄」概念は、人間存在の本質を個人と社会の二重性において捉える。この概念が世界の哲学的伝統においてどのような類縁概念を持つかを探索する事例を示す。

```r
library(philworkbench)

# 和辻『倫理学』コーパスの読み込み
corpus <- phil_read("watsuji_rinrigaku")

# 伝統横断的概念探索
results <- phil_explore(
  "人間の存在は本質的に他者との間柄においてある",
  traditions = "all",
  top_k = 30
)

# 伝統別マッピングの可視化
phil_plot(results, type = "tradition_map")
```

Listing 2　和辻「間柄」の伝統横断探索

`phil_explore()` は内部で以下の処理を実行する：(1) 入力テキストを philengine でファセット埋め込み（定義・用法・関係性の 3 ファセット）に変換、(2) PhilCorpusHub 上の全伝統コーパスに対してハイブリッド整列を実行、(3) 結果を PhilExploration オブジェクトとして返却。
期待される出力の上位結果を表 3 に示す。

表 3　「間柄」探索の期待される上位結果

順位	概念	伝統	スコア	言語
1	Mitsein（共存在）	ドイツ現象学	0.82	ドイツ語
2	Ich-Du（我と汝）	ユダヤ哲学	0.79	ドイツ語
3	Ubuntu	南アフリカ哲学	0.71	英語/ズールー語
4	仁	儒教	0.68	古典中国語
5	縁起	仏教（中観）	0.65	サンスクリット語
6	他者の顔	レヴィナス倫理学	0.63	フランス語
7	Dasein（現存在）	ドイツ現象学	0.59	ドイツ語
8	場所	京都学派	0.57	日本語

図 1 に、`phil_plot(results, type = "tradition_map")` が生成する伝統別マッピングの概念図を示す。各ノードは概念を表し、中心からの距離はスコア（小さいほど近い）、色は哲学的伝統を表す。

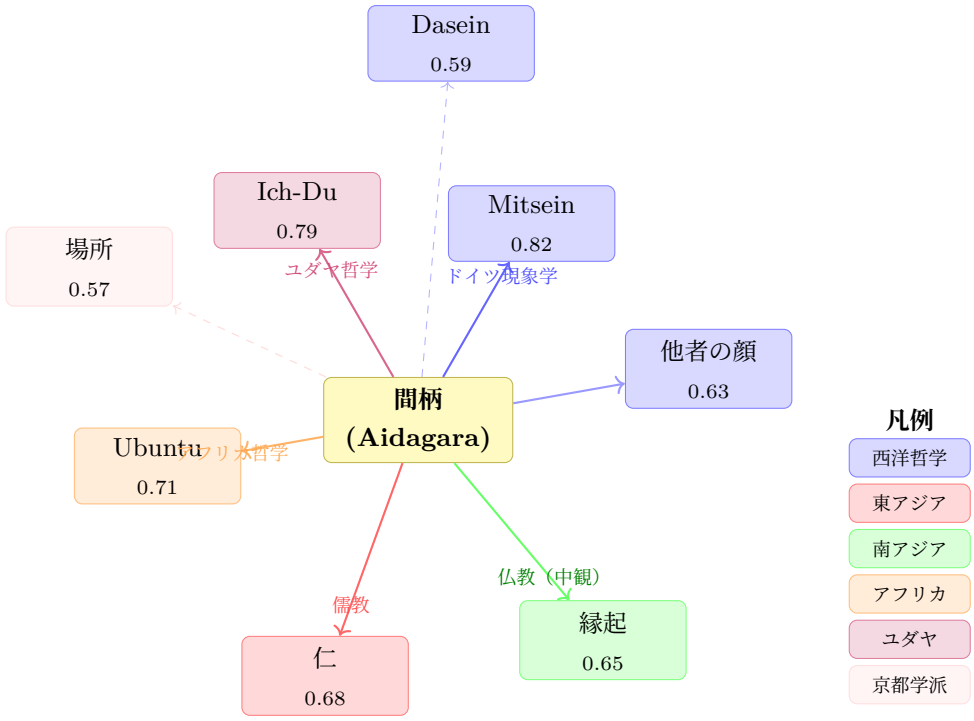

図 1　「間柄」の伝統横断探索結果マップ。中心からの距離はハイブリッドスコア（近いほど高類似度）、色は哲学的伝統を表す。

この結果は、以下の哲学的洞察を示唆する。第一に、Mitsein との高い類似度（0.82）は、和辻がハイデガーから直接的に影響を受けていることを反映している（和辻は 1927–1928 年にフライブルクでハイデガーの講義を聴講した）。第二に、Ubuntu との類似度（0.71）は、哲学史上の直接的接触がないにもかかわらず高い値を示しており、関係性に基づく人間存在の理解という構造的な類似を示唆する。この種の「意外な類縁関係」の発見こそが、計算的探索の最大の価値である。

4.2　事例 2：概念比較レポートの自動生成

探索で発見された候補概念を、多次元的に比較する事例を示す。

```
# 間柄とMitseinの多次元比較
comparison <- phil_compare(
  "間柄", "Mitsein",
  aspects = c("semantic", "structural",
              "argumentative", "historical")
)

# 比較結果の確認
print(comparison)

# LaTeX表の自動生成
phil_to_latex(comparison, style = "comparison_table")
```

Listing 3　概念比較レポートの自動生成

`phil_compare()` は各 aspect について異なる整列手法を適用する。`semantic` は多言語埋め込みの

余弦類似度、structural はフィロソフィカル・ナレッジグラフにおける位置比較、argumentative は両概念が出現する論証構造の比較、historical は影響関係・同時代性の評価を行う。
表 4 に、生成される比較結果の例を示す。

表 4　phil_compare("間柄", "Mitsein") の出力例

側面	スコア	説明
意味的	0.82	両概念とも人間存在の関係性次元を強調。ただし「間柄」は社会的・倫理的な関係に重点を置き、Mitsein は存在論的構造に重点を置く
構造的	0.74	各体系内で類似の位置を占める：個体の自立性と関係性の弁証法の結節点。ただし和辻体系では「空」との関連がある点で異なる
論証的	0.68	両概念とも個人主義的人間観への批判という論証的役割を共有。Mitsein はデカルト的主観性批判、間柄は近代的個人主義批判の文脈で使用
歴史的	0.91	和辻はハイデガーから直接的影響を受けており（1927–28 年フライブルク留学）、歴史的関係が実証されている
ハイブリッド	**0.79**	

phil_to_latex(comparison) は、この比較結果を学術論文に直接挿入可能な LaTeX コードとして出力する。これにより、計算的分析の結果を論文執筆に直接活用することができ、研究ワークフローの効率を大幅に向上させる。

4.3　事例 3：概念の時系列変遷追跡

哲学的概念は時代とともに意味が変化する。「自然」（nature / physis / 自然）概念がギリシャ哲学から現代に至るまでどのように変遷してきたかを追跡する事例を示す。

```r
# 概念の系譜追跡
genealogy <- phil_trace(
  "nature",
  type = "genealogy",
  periods = c("ancient_greek", "medieval",
              "early_modern", "modern", "contemporary")
)

# タイムライン可視化
phil_plot(genealogy, type = "timeline")
```

Listing 4　「自然」概念の時系列変遷追跡

図 2 に、期待される出力の概念図を示す。

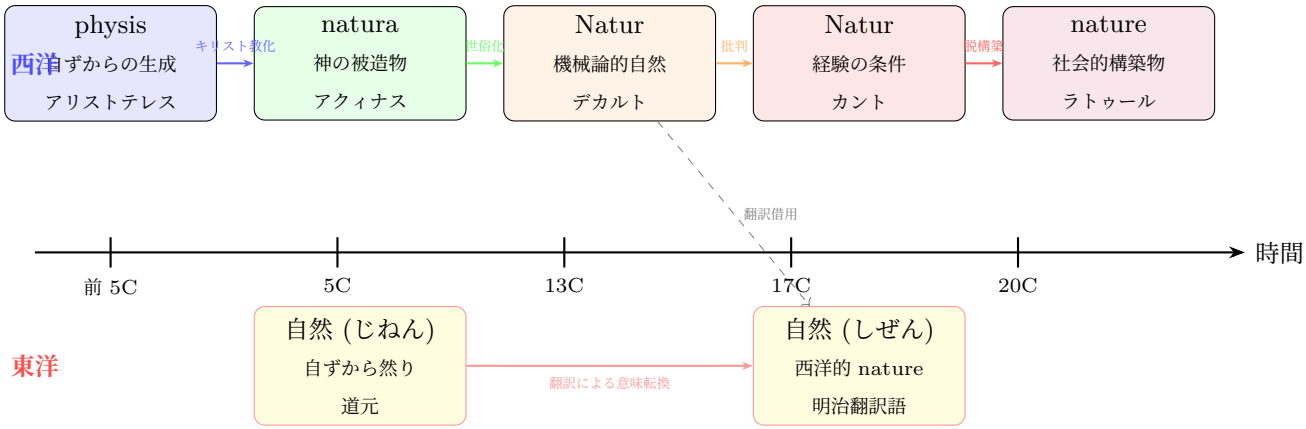

図 2　「自然」概念の時系列変遷。上段は西洋哲学における physis → natura → Natur → nature の意味変遷、下段は日本語における「自然」の意味転換（じねん → しぜん）を示す。

この可視化は、「自然」が古典ギリシャの physis（自ずからの生成・成長）から、中世のキリスト教化を経て神の被造物（natura naturata）となり、近代の機械論的自然観へ、そして現代の社会構築主義的理解へと変遷してきた過程を示す。東洋の並行展開として、日本語の「自然」（じねん）が道元における「自ずから然り」という仏教的意味から、明治期の翻訳を通じて西洋的 nature の意味に転換した過程も同時に表示される。

4.4　事例 4：解釈の布置図

同一テキストに対する複数の解釈を二次元空間に布置し、解釈間の距離と分布を可視化する事例を示す。ここではハイデガー『存在と時間』第 26 節の Mitsein 解釈を取り上げる。

```
# 解釈の収集と埋め込み
interpretations <- phil_read("heidegger_sz",
                             section = "26",
                             interpretations = "all")

# UMAP次元縮約と可視化
phil_plot(interpretations,
          type = "interpretation_map",
          method = "umap",
          color_by = "school",
          label_by = "annotator")
```

Listing 5　解釈の布置図

図 3 に、期待される出力の概念図を示す。

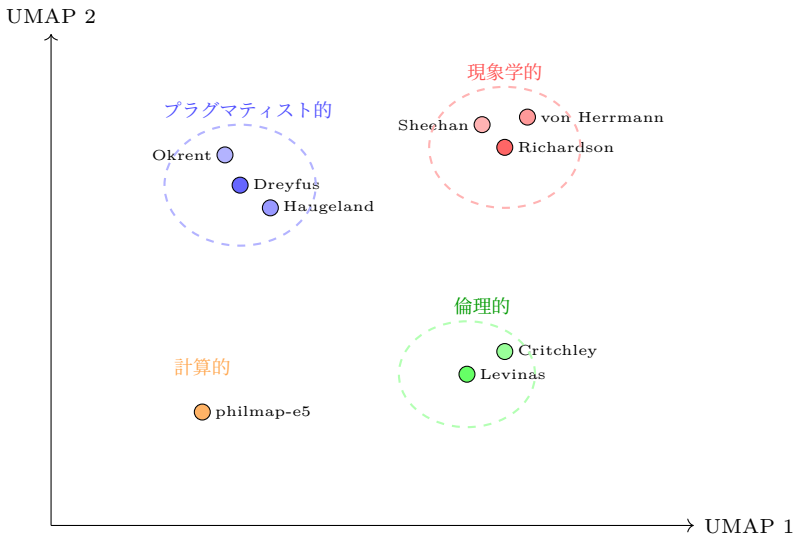

図 3　ハイデガー『存在と時間』§26（Mitsein）の解釈布置図（UMAP 概念図）。各点は一つの解釈を表し、色は解釈学派、空間的距離は解釈間の類似度を反映する。

この布置図は、Mitsein の解釈が大きく 3 つのクラスタ —— プラグマティスト的読み（Dreyfus, Haugeland, Okrent）、現象学的読み（Richardson, von Herrmann, Sheehan）、倫理的読み（Levinas, Critchley）—— に分かれることを示す。計算的読み（philmap-e5）は独立した位置を占めており、既存の解釈伝統とは異なる角度からテキストを捉えていることが示唆される。

4.5　事例 5：ゼミ・教育利用

Phil Ecosystem は研究だけでなく、教育場面でも活用できる。哲学ゼミにおいて、学生がテキストにアノテーションを付し、それを比較・議論する事例を示す。

```
# Shiny ダッシュボードの起動
phil_dashboard(
  corpus = phil_read("heidegger_sz"),
  mode = "seminar",
  participants = c("student_A", "student_B",
                   "student_C", "instructor")
)
```

Listing 6　ゼミでのインタラクティブ分析

`phil_dashboard(mode = "seminar")` は、以下の機能を持つ Shiny アプリケーションを起動する：

1. **テキストビューア**: テキストを表示し、概念アノテーションをオーバーレイ表示
2. **アノテーションパネル**: 各参加者が独立にテキストに概念タグを付与
3. **合意度表示**: 参加者間のアノテーション一致度をリアルタイムに計算・表示（Krippendorff の α）
4. **議論支援**: 不一致箇所をハイライトし、議論のポイントを自動抽出
5. **探索パネル**: 選択した概念の類似概念を他の伝統から探索

教育的効果として、(1) 学生がテキストの「どの部分」を「どの概念」として読むかの差異が可視化

されることで、解釈の多様性への気づきが促される、(2) 計算的探索により、学生が自分の知識の範囲外の伝統・概念に触れる機会が生まれる、(3) アノテーションの蓄積が後続のゼミのためのデータとなる、という利点がある。

4.6 事例 6：科研費申請・論文準備ワークフロー

Phil Ecosystem は、科研費申請書や学術論文の準備を支援する一連のワークフローを提供する。

```
# Step 1: 研究テーマに関する概念探索
results <- phil_explore(
  "間柄",
  traditions = c("western", "east_asian", "african"),
  top_k = 20
)

# Step 2: 有望な比較対を詳細に比較
comparison <- phil_compare(
  "間柄", "Ubuntu",
  aspects = c("semantic", "structural",
              "argumentative", "historical")
)

# Step 3: LaTeX表として出力（論文に直接挿入可能）
phil_to_latex(comparison, style = "comparison_table",
              file = "tables/aidagara_ubuntu.tex")

# Step 4: 再現性情報の自動記録
session <- phil_session()
phil_export(session, file = "provenance/session_info.json")
```

Listing 7　研究ワークフロー：探索から論文執筆まで

`phil_session()` は、使用したモデル（philmap-e5-v2)、バージョン（philworkbench 0.3.0)、計算パラメータ、コーパスバージョン、実行日時、乱数シードなどを自動的に記録する。これにより、計算的分析の完全な再現性が保証される。

科研費申請においては、`phil_explore()` の結果を「予備的調査」として示し、計算的アプローチにより「これまで認識されていなかった概念的類縁関係」が発見可能であることを具体的に示すことができる。再現性情報の自動記録は、学術論文におけるデータ・分析の透明性要件にも対応する。

5 データ構造：PhilCorpus と ConceptSpans

5.1 PhilCorpus：哲学テキストのための標準データコンテナ

PhilCorpus は、Phil Ecosystem のコアデータ構造であり、Bioconductor の SummarizedExperiment を哲学データに適応させたものである。図 4 に、その構造を示す。

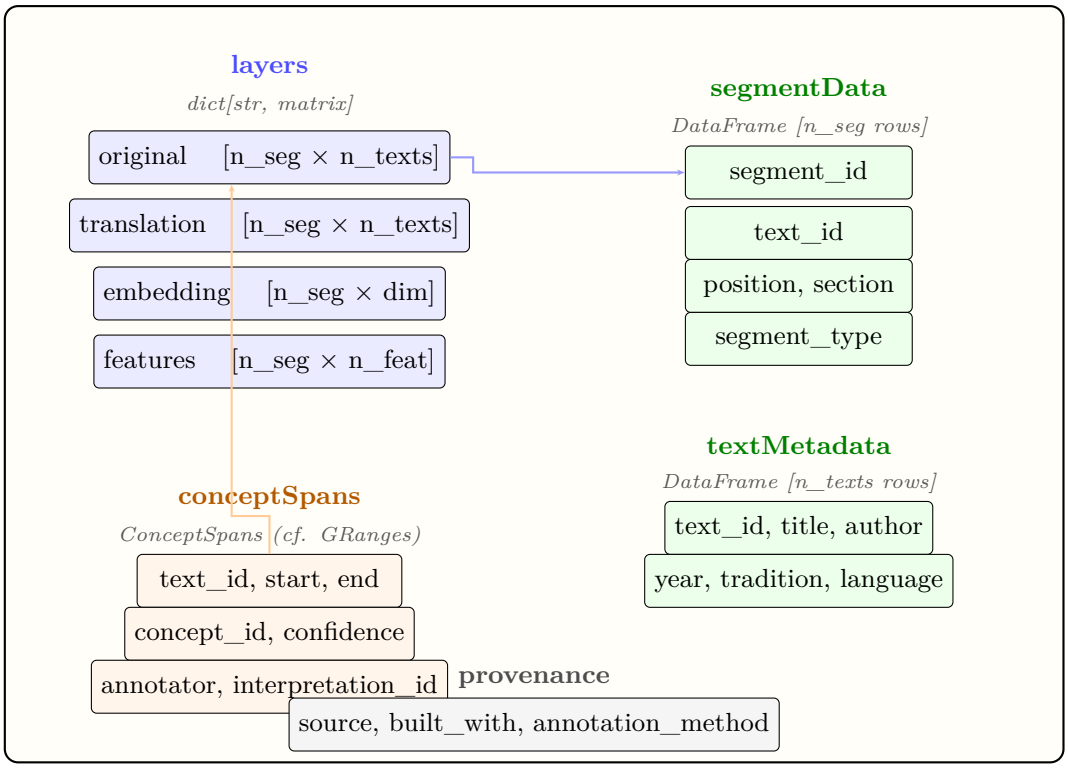

図 4　PhilCorpus のデータ構造。SummarizedExperiment の assays ↔ layers、rowData ↔ segmentData、colData ↔ textMetadata、rowRanges ↔ conceptSpans の対応関係を持つ。

SummarizedExperiment との構造的対応を詳しく説明する。

layers（assays 対応）は、同一のテキスト区間に対する複数の「表現」を格納する。ゲノミクスにおいて、同一遺伝子に対して生カウント、正規化カウント、TPM（Transcripts Per Million）という異なるアッセイが存在するように、哲学テキストにおいても原文、翻訳、埋め込みベクトル、特徴量ベクトルという異なる「層」が存在する。

segmentData（rowData 対応）は、テキストの各区間（段落、文、論証単位）に関するメタデータである。ゲノミクスにおける遺伝子の染色体位置、バイオタイプ等に対応する。区間の識別子、所属テキスト、テキスト内位置、章節情報、区間種別を含む。

textMetadata（colData 対応）は、各テキストに関するメタデータである。ゲノミクスにおけるサンプルの実験条件、患者情報等に対応する。テキスト識別子、タイトル、著者、年代、伝統、言語、版情報を含む。

conceptSpans（rowRanges / GRanges 対応）は、テキスト上の概念出現位置を記録するアノテーション構造である。GRanges がゲノム上の位置（染色体、開始位置、終了位置、ストランド）を記録するように、ConceptSpans はテキスト上の概念位置（テキスト ID、開始位置、終了位置、概念 ID）を記録する。これに加えて、信頼度、注釈者、解釈 ID を持つ点が GRanges との違いである。

R 版の PhilCorpus は S3 クラスとして実装され、SummarizedExperiment と同様のアクセサメソッドを提供する：

```
# コンストラクタ
corpus <- PhilCorpus(
  layers = list(
```

```
    original  = text_matrix,
    embedding = dense_matrix
  ),
  segmentData = DataFrame(
    segment_id = seg_ids,
    text_id = text_ids,
    position = positions
  ),
  textMetadata = DataFrame(
    text_id = text_ids,
    title = titles,
    author = authors,
    tradition = traditions
  ),
  conceptSpans = ConceptSpans(...)
)

# アクセサ（SE風）
layers(corpus, "embedding")   # 埋め込み層の取得
segmentData(corpus)           # 区間メタデータ
textMetadata(corpus)          # テキストメタデータ
conceptSpans(corpus)          # 概念スパン
corpus[i, j]                  # サブセット
```

Listing 8　PhilCorpus の R 版操作例

Python 版は dataclass として実装される：

```
@dataclass
class PhilCorpus:
    assay: np.ndarray           # [n_concepts x n_texts]
    concept_data: pd.DataFrame  # 概念メタデータ
    text_data: pd.DataFrame     # テキストメタデータ
    metadata: dict[str, Any]    # コーパスメタデータ

    @property
    def n_concepts(self) -> int:
        return self.assay.shape[0]

    @property
    def n_texts(self) -> int:
        return self.assay.shape[1]

    def subset_concepts(self, idx) -> PhilCorpus:
        return PhilCorpus(
            assay=self.assay[idx, :],
            concept_data=self.concept_data.iloc[idx],
            text_data=self.text_data.copy(),
            metadata=dict(self.metadata),
        )
```

Listing 9　PhilCorpus の Python 版

5.2　ConceptSpans：テキスト上の概念アノテーション

ConceptSpans は、Bioconductor の GRanges / IRanges に対応するテキスト上の概念位置アノテーション構造である。GRanges が「ゲノム上のどこに何があるか」を記述するように、ConceptSpans は「テキスト上のどこにどの概念が出現するか」を記述する。
GRanges との主な対応は以下の通りである：

- GRanges の `seqnames`（染色体名）↔ ConceptSpans の `text_id`（テキスト識別子）
- GRanges の `start/end`（位置）↔ ConceptSpans の `start/end`（文字位置）
- GRanges の `strand`（ストランド）↔ 該当なし
- GRanges の `mcols`（メタデータ列）↔ ConceptSpans の concept_id, confidence, annotator 等

GRanges にはない ConceptSpans 固有のフィールドとして、`annotator`（注釈者: 人間かモデルかの識別）と `interpretation_id`（解釈レイヤーの識別子）がある。これらは次節で述べる「解釈の多重性」を支える重要な要素である。

5.3　解釈の多重性への対応

SummarizedExperiment にない Phil Ecosystem 固有の設計要素が、解釈の多重性（interpretive multiplicity）の第一級サポートである。
ゲノムデータには基本的に一つの参照配列（reference genome）がある。GRCh38 上の chr1:1000-2000 という位置は、誰が解析しても同じゲノム領域を指す。しかし哲学テキストでは、同一のテキスト区間に対して複数の正当な解釈が並立しうる。
PhilCorpus はこの問題に対して、ConceptSpans の `interpretation_id` フィールドにより、同一テキスト区間に対する複数の解釈を格納する。

```r
# 利用可能な解釈の一覧
interpretations(corpus)
#>   interp_id  annotator    school            year
#> 1 I001       dreyfus      pragmatist        1991
#> 2 I002       richardson   phenomenological  1963
#> 3 I003       philmap-e5   computational     2026

# 特定の解釈に基づくアノテーション
conceptSpans(corpus, interpretation = "I001")

# 解釈間の一致度
interp_agreement(corpus, c("I001", "I002"))
#> Krippendorff's alpha: 0.43
```

Listing 10　解釈の多重性の操作

この設計は、以下の研究課題を支える：(1) 異なる解釈伝統がどの程度一致・相違するかの定量的評価、(2) 計算的解釈（モデルによる自動アノテーション）と人間の解釈の比較、(3) 解釈の変遷の時系列追跡。

6　エコシステムアーキテクチャ

本章以降は、インフォマティクス研究者を主な読者として、Phil Ecosystem の技術的詳細を記述する。

6.1　7 層アーキテクチャ

Phil Ecosystem は 7 層のアーキテクチャを採用している（図 5）。各層は明確な責務を持ち、上位層は下位層に依存するが、逆方向の依存は存在しない。

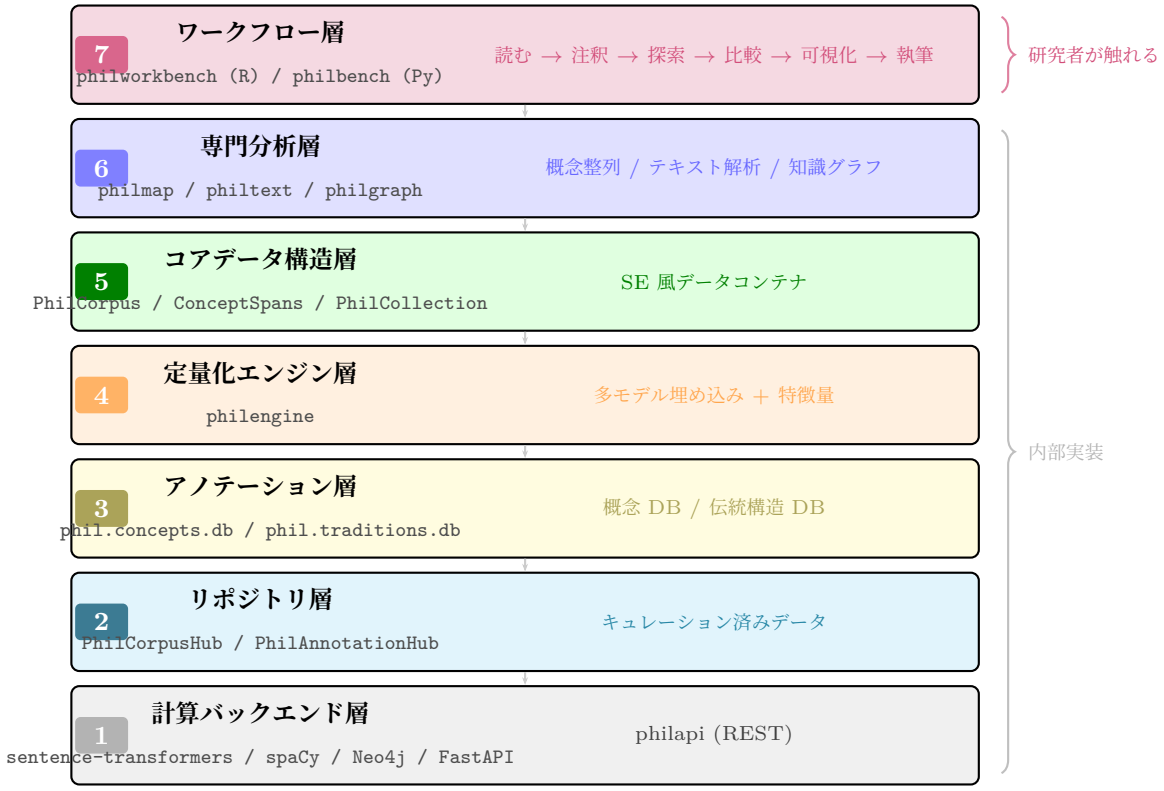

図 5　Phil Ecosystem の 7 層アーキテクチャ。研究者は最上位のワークフロー層のみを操作し、内部の 6 層を意識する必要がない。

各層の責務と主要コンポーネントは以下の通りである。

Layer 1: 計算バックエンド層は、外部の ML/NLP ライブラリ（sentence-transformers, spaCy, Neo4j 等）と FastAPI による REST API サーバーを含む。この層は直接ユーザーに公開されず、上位層から API 経由でアクセスされる。

Layer 2: リポジトリ層は、Bioconductor の ExperimentHub / AnnotationHub に対応し、キュレーション済みの PhilCorpus とアノテーションリソースを提供する。

Layer 3: アノテーション層は、Bioconductor の org.Hs.eg.db / TxDb.* に対応し、概念アノテー

ション（phil.concepts.db）と伝統構造（phil.traditions.db）の SQLite データベースを提供する。
Layer 4: 定量化エンジン層（philengine）は、テキストを数値表現に変換する統一エンジンである。プラガブルバックエンド設計により、sentence-transformers, OpenAI, Cohere 等を切り替え可能。
Layer 5: コアデータ構造層は、PhilCorpus, ConceptSpans, PhilCollection のデータコンテナを定義する。
Layer 6: 専門分析層は、概念整列（philmap）、テキスト解析（philtext）、知識グラフ（philgraph）の 3 つのドメイン特化パッケージを含む。
Layer 7: ワークフロー層は、約 15 の動詞（phil_read, phil_explore 等）を提供するファサードパッケージであり、研究者が直接操作する唯一の層である。

6.2 R/Python 二言語戦略

Phil Ecosystem は、R と Python の二言語を対等に扱う戦略を採用している。この決定は、デジタル・ヒューマニティーズ研究者と機械学習研究者という 2 つの主要なユーザー層に対応するためである。
計算層（埋め込み計算、NLP 処理、知識グラフクエリ）は Python で実装される。これは、sentence-transformers, spaCy, Neo4j Python ドライバなどの主要な ML/NLP ライブラリが Python エコシステムに存在するためである。
インターフェース層（データ操作、可視化、ワークフロー）は R と Python の両方で提供される。R 版は ggplot2, shiny, tidyverse との統合が強みであり、Bioconductor に馴染みのある DH 研究者に自然なインターフェースを提供する。
R パッケージ群は、philapi（FastAPI サーバー）を経由して Python の計算層にアクセスする。型の一貫性は、`shared/schemas/` ディレクトリに配置された JSON Schema ファイルにより保証される。

```
# shared/schemas/concept.schema.json
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Concept",
  "type": "object",
  "properties": {
    "concept_id": {"type": "string",
                   "pattern": "^phil:C[0-9]+$"},
    "labels": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "text": {"type": "string"},
          "language": {"type": "string"}
        }
      }
    },
    "tradition": {"type": "string"},
    "era": {"type": "string"}
```

```
  }
}
```

Listing 11　JSON Schema による型定義の共有

6.3　リポジトリ構成

Phil Ecosystem は、メインモノレポとの複数の衛星リポジトリで構成される。表 5 にリポジトリ一覧を示す。

表 5　リポジトリ構成一覧

リポジトリ	役割	パッケージ数	主要技術
phil/	メインモノレポ	Py:7, R:6	Python, R
phil-hub/	データリポジトリ基盤	2×2	S3/GCS, YAML
phil-texts/	正典テキストパッケージ	可変	R data packages
phil-models/	学習済みモデル	–	HuggingFace Hub

メインモノレポ `phil/` の構造は以下の通りである：

- `python/` — Python パッケージ群(philcore, philengine, philmap, philtext, philgraph, philbench, philapi)
- `r/` — R パッケージ群（philcore, philengine, philmap, philworkbench, phil.concepts.db, phil.traditions.db）
- `shared/` — 言語共通リソース（JSON Schema, オントロジー定義, ベンチマークデータ, 設定ファイル）
- `scripts/` — ビルド・ファインチューニングスクリプト
- `docker/` — Docker 構成（API + Neo4j + Redis）
- `tests/` — 統合テスト（R↔Python ラウンドトリップ、E2E パイプライン）

6.4　依存関係グラフ

図 6 に、パッケージ間の依存関係を有向非巡回グラフ（DAG）として示す。

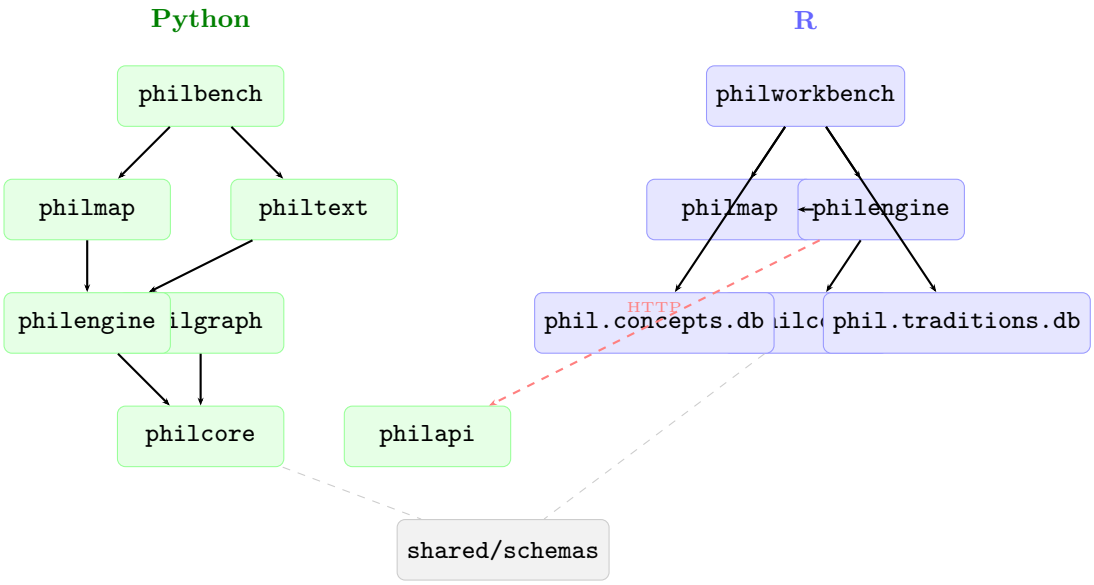

図 6　パッケージ依存関係の DAG。実線は直接依存、赤い破線は REST API 経由の言語間接続、灰色の破線は JSON Schema の共有を表す。

7　テキスト定量化エンジン（philengine）

7.1　プラガブルバックエンド設計

philengine は、テキストを数値表現に変換する統一エンジンであり、プラガブルバックエンド設計を採用している。すべてのバックエンドは EmbeddingBackend プロトコルに準拠する：

```
class EmbeddingBackend(Protocol):
    def encode(self, texts: list[str],
               **kwargs) -> np.ndarray: ...
    def dim(self) -> int: ...
    def max_tokens(self) -> int: ...
    def metadata(self) -> BackendMetadata: ...
```

Listing 12　EmbeddingBackend プロトコル

表 6 に、利用可能なバックエンドの一覧を示す。

表 6　埋め込みバックエンド一覧

バックエンド	実装	主な用途	次元
SentenceTransformers	sentence-transformers	ローカル推論（主力）	768–1024
OpenAI	OpenAI API	text-embedding-3-large	3072
Cohere	Cohere API	embed-multilingual-v3	1024
HuggingFace	HF Inference API	リモート推論	可変
LocalLLM	vLLM / llama.cpp	自前サーバー	可変
Cached	ラッパー	キャッシュ追加	–

バックエンドの切り替えは、研究者のコードを変更することなく、設定レベルで行うことができる：

```r
# ローカルモデル
engine_local <- phil_engine(
  backend = "sentence-transformers",
  model = "philmap-e5-finetuned-v2"
)

# OpenAI API
engine_api <- phil_engine(
  backend = "openai",
  model = "text-embedding-3-large"
)

# 同一のインターフェースで利用
emb_local <- phil_embed(engine_local, texts)
emb_api   <- phil_embed(engine_api, texts)

# バックエンド間のベンチマーク比較
phil_compare_backends(
  list(engine_local, engine_api),
  benchmark = "concept_pairs_v2",
  metrics = c("spearman", "precision_at_k")
)
```

Listing 13　バックエンドの切り替え

7.2　ファセット埋め込み

哲学的概念は、単一のベクトルでは十分に表現できない多面的な意味を持つ。philengine は、概念を 3 つのファセット（側面）で別々にベクトル化し、重み付き結合で複合ベクトルを生成する「ファセット埋め込み」を提供する。

- **Definition**（定義ファセット）：概念の意味的内容を捉える。「間柄とは何か」
- **Usage**（用法ファセット）：概念が論証やテキストでどのように使われるかを捉える。「間柄はどのような文脈で使用されるか」
- **Relational**（関係性ファセット）：概念が属する伝統内のオントロジーにおける位置を捉える。「間柄は和辻倫理学の体系内でどこに位置するか」

図 7 に、ファセット埋め込みの概念図を示す。

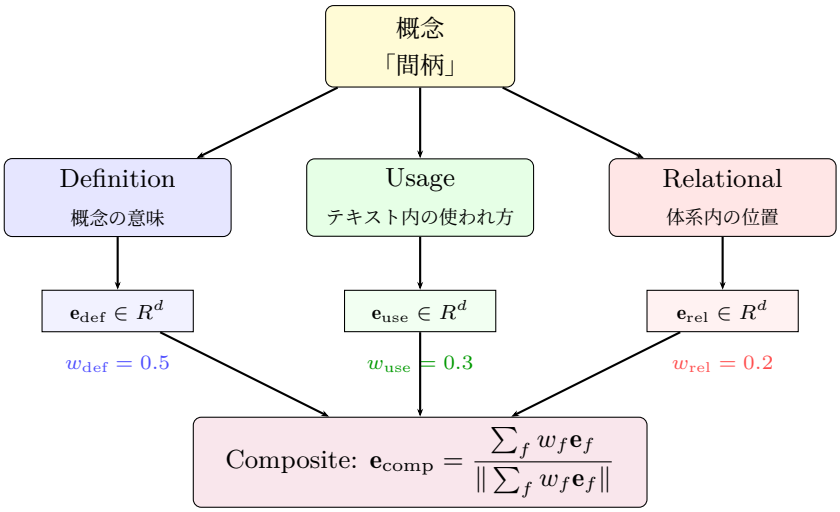

図 7　ファセット埋め込みの概念図。概念を 3 つの側面（定義・用法・関係性）で個別にベクトル化し、重み付き結合で複合ベクトルを生成する。

ファセット埋め込みの数式を以下に示す。概念 c の各ファセット $f \in \{\text{def}, \text{use}, \text{rel}\}$ に対して、埋め込みモデル ϕ でベクトル化する：

$$\mathbf{e}_f(c) = \phi(\text{prompt}_f(c)) \in R^d \tag{1}$$

ここで $\text{prompt}_f(c)$ は、ファセット f に応じたプロンプトテンプレートで生成されたテキストである。複合ベクトルは重み付き結合と正規化で得られる：

$$\mathbf{e}_{\text{comp}}(c) = \frac{\sum_f w_f \cdot \mathbf{e}_f(c)}{\left\|\sum_f w_f \cdot \mathbf{e}_f(c)\right\|} \tag{2}$$

デフォルトの重みは $w_{\text{def}} = 0.5$, $w_{\text{use}} = 0.3$, $w_{\text{rel}} = 0.2$ であるが、研究目的に応じて変更可能である。

実装は以下の通りである：

```python
@dataclass
class FacetedEmbedding:
    definition: np.ndarray
    usage: np.ndarray
    relational: np.ndarray
    config: EmbeddingConfig = field(
        default_factory=EmbeddingConfig
    )

    def composite(self,
                  weights: dict[str, float] | None = None
                  ) -> np.ndarray:
        w = weights or self.config.facet_weights
        vec = (w["definition"] * self.definition +
               w["usage"] * self.usage +
               w["relational"] * self.relational)
```

```
        norm = np.linalg.norm(vec)
        return vec / norm if norm > 1e-9 else vec
```

Listing 14　FacetedEmbedding の実装

7.3　非ベクトル的定量化

埋め込みベクトルだけでは捉えきれないテキストの特徴がある。philengine は、4 種類の非ベクトル的特徴量を提供する。

表 7　非ベクトル的特徴量一覧

カテゴリ	特徴量	説明
語彙・修辞	TTR	Type-Token Ratio（語彙の豊かさ）
	argument_marker_freq	論証マーカー（therefore, hence, 故に）の頻度
	citation_density	引用密度
	technical_term_ratio	専門語比率
構造的	argument_depth	論証の入れ子深さ
	premise_count	前提の数
	counterarg_structure	反論の有無と構造
	dialectical_pattern	弁証法パターン（正・反・合）
間テキスト	similarity_distribution	他テキストとの類似度分布
	novelty_score	当該テキストの新規性
	tradition_position	伝統内での位置（中心性・周縁性）

7.4　古典語への対応

哲学テキストは、現代語だけでなく古典語で書かれたものも多い。philengine は、古典ギリシャ語、漢文、サンスクリット語、古典ラテン語に対する前処理パイプラインを提供する：

```
preprocessor = PhilPreprocessor(
    classical_greek="cltk_normalize",
    classical_chinese="kanbun_segment",
    sanskrit="indic_transliterate",
    middle_japanese="nkf_normalize",
    classical_latin="cltk_lemmatize"
)
```

Listing 15　古典語前処理

古典ギリシャ語については CLTK（Classical Language Toolkit）による正規化とレンマ化、漢文については kanbun ライブラリによる文分割と訓読変換、サンスクリット語についてはデーヴァナーガリー文字から IAST（International Alphabet of Sanskrit Transliteration）への翻字を行う。
これらの前処理は、多言語埋め込みモデルが古典語テキストを適切に処理するための前段階として不可欠である。特に漢文については、空白なしの連続文字列を意味のある単位に分割する処理が、埋め

込みの品質に大きな影響を与える。

7.5 哲学ドメイン特化ファインチューニング

汎用の多言語埋め込みモデルは、哲学テキストの意味的類似性を十分に捉えられないことがある。例えば、「空」（śūnyatā）と「無」（Nichts）は日常言語では類似しないが、哲学的文脈では深い関連を持つ。

Phil Ecosystem では、multilingual-e5-base[13] をベースモデルとして、哲学的概念ペアのアノテーションデータでファインチューニングした philmap-e5 モデルを提供している。表 8 に、ファインチューニングの結果を示す。

表 8　哲学ドメイン特化ファインチューニングの結果

モデル	ベース	学習ペア数	Spearman ρ	備考
multilingual-e5-base	–	–	0.621	ベースライン
philmap-e5-v1	e5-base	94	0.845	初期版
philmap-e5-v2	e5-base	138	0.976	改善版

ベースラインの multilingual-e5-base では Spearman 相関 0.621 であったものが、94 ペアでのファインチューニング（v1）で 0.845 に、138 ペアへの拡張（v2）で 0.976 に向上した。この劇的な改善は、少数の高品質なアノテーションデータがドメイン特化の埋め込みに大きな効果を持つことを示している。

学習データは、哲学の学術文献に基づいて概念ペアの類似度を専門家がアノテーションしたものである。各ペアには 0（無関係）から 1（同義）の連続値スコアが付与されている。例えば、「間柄」–「Mitsein」ペアには 0.82、「空」–「Nichts」には 0.55、「ロゴス」–「道」には 0.48 が付与されている。

8 概念整列手法（philmap）

8.1 4 つの整列手法

philmap は、異なる哲学的伝統に属する概念を多次元的に比較するための 4 つの整列手法を提供する。各手法は異なる側面の類似性を捉える。

8.1.1 意味的整列（Semantic Alignment）

多言語埋め込みモデルによる余弦類似度を計算する：

$$s_{\text{sem}}(c_1, c_2) = \frac{\mathbf{e}(c_1) \cdot \mathbf{e}(c_2)}{\|\mathbf{e}(c_1)\| \cdot \|\mathbf{e}(c_2)\|} \tag{3}$$

ファセット埋め込みを使用する場合、各ファセットの類似度を個別に計算し、重み付き平均をとる：

$$s_{\text{sem}}^{\text{facet}}(c_1, c_2) = \sum_f w_f \cdot \cos(\mathbf{e}_f(c_1), \mathbf{e}_f(c_2)) \tag{4}$$

8.1.2 構造的整列（Structural Alignment）

知識グラフ（philgraph）上での位置的類似性を比較する。二つの概念が、それぞれの伝統内で類似した構造的位置を占めるか（例：上位概念の数、下位概念の数、関連概念のパターンが類似しているか）をグラフ理論的指標で評価する：

$$s_{\text{str}}(c_1, c_2) = \alpha \cdot \text{sim}_{\text{degree}}(c_1, c_2) + \beta \cdot \text{sim}_{\text{centrality}}(c_1, c_2) + \gamma \cdot \text{sim}_{\text{motif}}(c_1, c_2) \tag{5}$$

ここで $\text{sim}_{\text{degree}}$ は次数分布の類似度、$\text{sim}_{\text{centrality}}$ は中心性指標の比較、$\text{sim}_{\text{motif}}$ は局所グラフモチーフの一致度を表す。

8.1.3 論証的整列（Argumentative Alignment）

両概念が哲学的論証において果たす役割の類似性を評価する。例えば、「間柄」と「Mitsein」は共に「個人主義的人間観への批判」という論証的役割を共有する：

$$s_{\text{arg}}(c_1, c_2) = \frac{|R(c_1) \cap R(c_2)|}{|R(c_1) \cup R(c_2)|} \tag{6}$$

ここで $R(c)$ は概念 c が出現する論証における役割の集合（前提、結論、反論対象、支持論拠など）である。

8.1.4 ハイブリッド整列（Hybrid Alignment）

3 つの整列手法を重み付き結合する：

$$s_{\text{hybrid}}(c_1, c_2) = w_{\text{sem}} \cdot s_{\text{sem}} + w_{\text{str}} \cdot s_{\text{str}} + w_{\text{arg}} \cdot s_{\text{arg}} \tag{7}$$

デフォルトの重みは $w_{\text{sem}} = 0.4$, $w_{\text{str}} = 0.35$, $w_{\text{arg}} = 0.25$ である。
実装は以下の通り：

```python
class HybridAlignment(AlignmentMethod):
    def __init__(self, semantic, structural,
                 argumentative, weights=None):
        self.semantic = semantic
        self.structural = structural
        self.argumentative = argumentative
        self.weights = weights or {
            "semantic": 0.4,
            "structural": 0.35,
            "argumentative": 0.25,
        }

    def align(self, source, target):
        m_sem = self.semantic.align(source, target)
        m_str = self.structural.align(source, target)
        m_arg = self.argumentative.align(source, target)
        score = (
            self.weights["semantic"] * m_sem.overall_score
            + self.weights["structural"]
```

```
                * m_str.overall_score
            + self.weights["argumentative"]
                * m_arg.overall_score
        )
        return ConceptMapping(
            source=source, target=target,
            overall_score=score,
            alignment_type=AlignmentType.HYBRID,
            evidence=(m_sem.evidence
                      + m_str.evidence
                      + m_arg.evidence),
        )
```

Listing 16　HybridAlignment の実装

8.2　ベンチマーク

ファインチューニング済みモデルの性能を評価するため、15 の概念ペアからなるベンチマークデータセットを構築した。各ペアには、哲学の専門家による 0–1 の類似度スコアが ground truth として付与されている。

表 9: 概念ペアベンチマーク（15 ペア）

概念 A	**概念 B**	**専門家スコア**	**計算スコア**
間柄	Mitsein	0.85	0.82
Ich-Du	Ubuntu	0.70	0.68
空（śūnyatā）	絶対無	0.75	0.73
仁	agape	0.55	0.52
道	logos	0.50	0.48
Dasein	現存在	0.95	0.94
カルマ	因果応報	0.60	0.58
アートマン	Seele	0.45	0.47
無為	Gelassenheit	0.55	0.51
li（理）	eidos	0.40	0.43
dharma	法	0.80	0.78
nirvāṇa	涅槃	0.92	0.91
aporeia	公案	0.35	0.38
phronēsis	実践知	0.70	0.67
Aufhebung	止揚	0.90	0.89

専門家スコアと計算スコアの間の Spearman 順位相関は $\rho = 0.976$（$p < 0.001$）であり、ファインチューニング済みモデルが哲学的概念の類似性を高い精度で捕捉していることを示す。

9　知識グラフとテキスト解析

9.1　philgraph：哲学知識グラフ

philgraph は、全伝統横断の哲学知識グラフを構築・クエリするためのパッケージである。図 8 に、グラフスキーマを示す。

9.1.1　ノード型

philgraph は 8 つのノード型を定義する：

1.**Concept** — 哲学的概念（例：間柄、Dasein、śūnyatā）
2.**Thinker** — 哲学者（例：和辻哲郎、ハイデガー、龍樹）
3.**Text** — 哲学テキスト/著作（例：『倫理学』『存在と時間』『中論』）
4.**Tradition** — 哲学的伝統/学派（例：京都学派、現象学、中観派）
5.**Argument** — 哲学的論証（例：超越論的演繹、帰謬法）
6.**Era** — 時代区分（例：古代、中世、近代、現代）
7.**Institution** — 学術機関（例：京都大学、フライブルク大学）
8.**Language** — 言語（例：日本語、ドイツ語、サンスクリット語）

9.1.2　エッジ型

15 のエッジ型が定義され、各エッジ型には許可されるノード型のペアが制約として設定されている：

1.**influences**: 影響関係（Thinker→Thinker, Concept→Concept, Tradition→Tradition）
2.**opposes**: 対立関係（Concept→Concept, Thinker→Thinker）
3.**extends**: 拡張関係（Concept→Concept, Argument→Argument）
4.**reinterprets**: 再解釈関係（Thinker→Concept）
5.**translates_to**: 翻訳関係（Concept→Concept）
6.**analogous_to**: 類縁関係（Concept→Concept）
7.**subsumes**: 包含関係（Concept→Concept）
8.**uses_in_argument**: 論証内使用（Argument→Concept）
9.**authored_by**: 著者関係（Text→Thinker）
10.**belongs_to_tradition**: 伝統帰属（Thinker→Tradition, Concept→Tradition）
11.**contemporary_with**: 同時代関係（Thinker→Thinker）
12.**affiliated_with**: 所属関係（Thinker→Institution）
13.**written_in**: 言語関係（Text→Language）
14.**part_of_era**: 時代帰属（Thinker→Era, Tradition→Era）
15.**cites**: 引用関係（Text→Text）

各エッジには、信頼度（0–1）、証拠ソース、合意レベル（established / debated / speculative / novel）のメタデータが付与される。

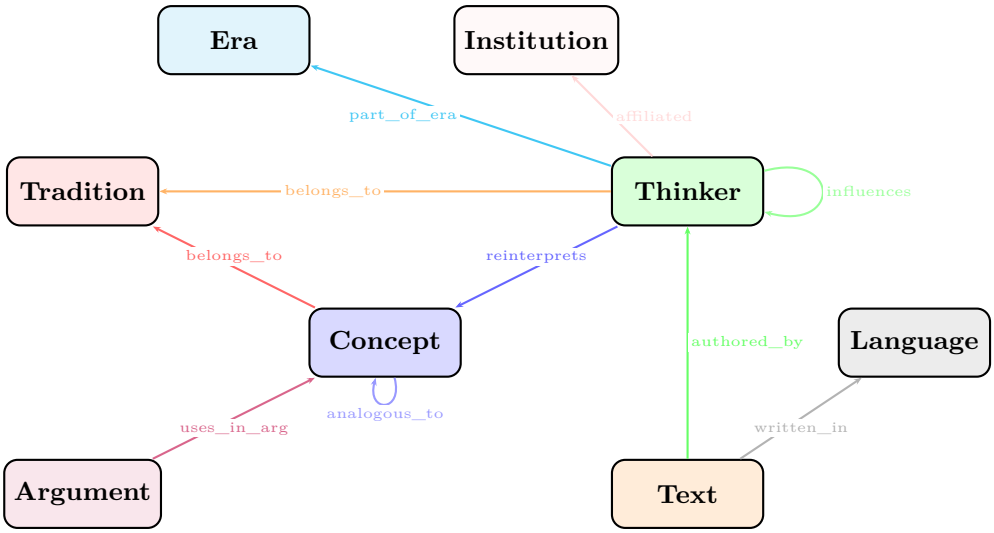

図 8　philgraph のグラフスキーマ。8 ノード型と主要なエッジ型の一部を示す。

9.2　philtext：論証抽出と解釈追跡

philtext は、哲学テキストに対するドメイン特化型 NLP パッケージであり、7 つのコンポーネントを含む。

表 10　philtext のコンポーネント

コンポーネント	主要クラス	機能
argument	ArgumentExtractor	論証抽出（ルールベース + LLM + ハイブリッド）
concept	ConceptExtractor	概念抽出・リンキング（辞書 + embedding）
classify	SchoolClassifier	学派分類（40+ 学派、prototype/NLI）
influence	InfluenceDetector	影響検出（dense retrieval + reranking）
hermeneutic	InterpretationTracker	解釈追跡・比較・衝突検出
corpus	CorpusBuilder	コーパス管理・多言語処理
bridge	PracticalMapper	実践的応用への橋渡し

SchoolClassifier は、40 以上の哲学学派を分類する。分類体系は 4 つの大伝統に階層化されている：

- **Western**（18 学派）：Presocratic, Platonic, Aristotelian, Stoic, Epicurean, Neoplatonic, Scholastic, Rationalist, Empiricist, Kantian, German Idealism, Phenomenology, Existentialism, Analytic, Pragmatism, Critical Theory, Poststructuralism, Process Philosophy
- **East Asian**（8 学派）：Confucian, Daoist, Legalist, Mohist, Chan/Zen Buddhist, Neo-Confucian, Kyoto School, New Confucianism
- **South Asian**（9 学派）：Nyaya, Vaisheshika, Samkhya, Yoga, Mimamsa, Vedanta, Buddhist (Madhyamaka), Buddhist (Yogacara), Jain
- **Islamic**（4 学派）：Kalam, Falsafa, Sufi Philosophy, Illuminationist

9.3　非古典論理サポート

Phil Ecosystem は、西洋の古典論理（二値論理）だけでなく、4 つの非古典論理体系を第一級にサポートする。これは、非西洋哲学を西洋的カテゴリに押し込めることなく、各伝統固有の論理的フレームワークで推論を記述するためである。

表 11　サポートされる非古典論理体系

論理体系	起源	値域	特徴
四句分別	龍樹（中観派）	4 値 + 全否定	肯定・否定・両方・いずれでもない。全否定が空を指示
場所論理	西田幾多郎	3 段階	有の場所 ⊂ 相対無 ⊂ 絶対無。自覚の構造
矛盾許容	Belnap	4 値	$\{T, F, Both, Neither\}$。矛盾を許容する推論
弁証法	ヘーゲル	3 項	テーゼ – アンチテーゼ – ジンテーゼ。止揚（Aufhebung）

これらの論理体系は philcore の ontology モジュールに実装されている。例えば、四句分別（catuskoti）の実装は次の通りである：

```python
class Koti(int, Enum):
    AFFIRMATION = 1    # 肯定
    NEGATION = 2       # 否定
    BOTH = 3           # 両肯定
    NEITHER = 4        # 両否定

class CatuskotiEvaluation(BaseModel):
    proposition: str
    koti_values: dict[Koti, bool | None]
    all_rejected: bool = False  # 龍樹の全否定
    commentary: str | None = None
```

Listing 17　四句分別（Catuskoti）の実装

四句分別の `all_rejected = True` は、龍樹の『中論』[4] における prasajya-pratisedha（帰謬否定）に対応し、4 つのすべての koti が否定されることで空（śūnyatā）が指し示される。これは二値論理では表現不可能な推論構造であり、非古典論理の第一級サポートが必要となる典型的な例である。

10　深層学習基盤との関係

10.1　大規模言語モデルとの連携

Phil Ecosystem は、大規模言語モデル（LLM）を哲学テキスト分析の補助ツールとして活用する。具体的な活用場面は 3 つある。

第一に、**論証抽出**への活用である。philtext の ArgumentExtractor は、ルールベースの抽出に加えて、LLM ベースの論証抽出を提供する。LLM は複雑な論証構造（入れ子の反論、暗黙の前提）の認

識に優れており、ルールベースでは捉えにくいテキスト内の論証パターンを補完的に発見できる。
第二に、**概念分類**への活用である。SchoolClassifier の NLI（Natural Language Inference）モードは、ゼロショットでの学派分類を可能にする。学習データが十分でない新たな学派の分類において有用である。
第三に、**解釈の生成**への活用である。計算的解釈（`interpretation_id = "philmap-e5"`）として、LLM を用いたテキスト解釈を生成し、人間の解釈と並置して比較することが可能である。これは LLM が「正しい」解釈を提供するという意味ではなく、計算的手法が生成する解釈と人間の解釈の差異を可視化するというメタ解釈学的な意義を持つ。
プロンプトエンジニアリング（既存 LLM の指示による活用）とファインチューニング（ドメイン特化の学習）は、タスクに応じて使い分ける。埋め込み生成のようなボトムレベルのタスクにはファインチューニング（philmap-e5-v2）が適し、論証抽出のような高レベルのタスクにはプロンプトエンジニアリングが現時点では実用的である。

10.2　多言語埋め込みモデルの発展と哲学への応用

近年の多言語埋め込みモデルの発展は、哲学テキストの計算的処理に大きな可能性を開いている。
E5[13]（Embeddings from bidirectional Encoder representations, 5 つのタスク向け）は、instruction-following embedding として、タスクに応じた指示プレフィックス（”query: ”, ”passage: ”）により埋め込みの用途を制御できる点が特徴である。Phil Ecosystem のデフォルトモデルとして採用されている multilingual-e5-large-instruct は、100 以上の言語に対応し、1024 次元のベクトルを生成する。
mDeBERTa-v3[15] は、disentangled attention mechanism によりトークンの内容と位置を分離して処理することで、長い文脈にわたる意味的関係の捕捉に優れている。philtext の NLI ベースの学派分類に使用されている。
哲学テキストにおけるこれらのモデルの課題として、以下が挙げられる：（1）古典語（古典ギリシャ語、サンスクリット語、漢文）のトークン化品質が現代語に比べて低い。前処理パイプライン（第 7 章 7.4 節）で部分的に対処するが、古典語特化のトークナイザの開発が今後の課題である。（2）哲学的専門用語の意味が日常語と大きく異なる場合（例：「存在」「本質」「実存」）、汎用モデルは日常的意味に偏る傾向がある。ファインチューニング（第 7 章 7.5 節）で対処するが、学習データの規模拡大が課題である。（3）概念の多義性 —— 同一の語が文脈により異なる概念を指す場合（例：「自然」のフシス的意味とネイチャー的意味）—— への対処は、文脈付き埋め込み（contextualized embeddings）によりある程度可能だが、解釈レイヤーとの統合が今後の課題である。

10.3　今後のモデル開発の方向性

Phil Ecosystem の今後のモデル開発は、3 つの方向性を持つ。
第一に、**解釈コンテキスト付き埋め込み**の開発である。現在の埋め込みモデルは、テキストの「意味」をベクトル化するが、「誰がどの学派の視点で読んだか」という解釈的コンテキストは考慮しない。今後は、解釈レイヤーの情報をプロンプトとして埋め込みモデルに入力することで、解釈依存の埋め込みを生成する手法を開発する。
第二に、**概念の生態学（ecology of concepts）**フレームワークの構築である。概念を孤立した実

体としてではなく、互いに影響し合い、競合し、共進化する「生態系」として捉える計算モデルを開発する。これは、概念の時間的変遷（第 4 章 4.3 節）と構造的関係（第 9 章）を統合するメタ理論的フレームワークである。
第三に、**哲学推論の計算モデル**の構築である。現在の Phil Ecosystem は概念の「比較」に焦点を当てているが、将来的には哲学的「推論」 —— ある概念的前提からどのような結論が導かれるか —— の計算的モデリングを目指す。非古典論理サポート（第 9 章 9.3 節）はこの方向性の基盤となる。

11　関連研究

11.1　デジタル・ヒューマニティーズ

デジタル・ヒューマニティーズ（DH）は、人文学研究に情報技術を導入する学際的分野であり、1990 年代以降急速に発展してきた [19]。テキストマイニング、トピックモデリング [20]、感情分析、スタイロメトリーなどの手法が文学研究・歴史学研究に広く応用されている。
しかし、DH の手法は主に大規模テキストの統計的分析を志向しており、哲学テキストの持つ「概念の深さ」「論証の構造」「解釈の多重性」を十分に捉える枠組みは提供していない。Phil Ecosystem は、DH の基盤技術（コーパス処理、テキストマイニング）を活用しつつ、哲学固有の要件（概念整列、解釈追跡、非古典論理）に特化した機能を提供する点で、従来の DH ツールを補完する。

11.2　計算哲学

計算哲学（Computational Philosophy）は、形式論理学、計算理論、人工知能の手法を哲学的問題の分析に応用する分野である。Thagard の概念変化の計算理論 [21]、Leitgeb の形式認識論 [22]、Williamson のモデル論的方法論 [23] などが代表的な研究である。
Phil Ecosystem は、これらの研究がトップダウンに（形式理論から哲学テキストへ向かって）哲学的問題にアプローチするのに対し、ボトムアップに（哲学テキストのデータから概念的構造を抽出する方向に）アプローチする点で異なる。両者は相補的であり、Phil Ecosystem が発見した概念的構造を計算哲学の形式理論で検証するという連携が可能である。

11.3　哲学的テキストマイニング

PhilPapers[16] は現代英語圏の哲学論文の包括的なデータベースであり、分類体系と検索機能を提供する。InPhO[17] は哲学的概念のオントロジーであり、概念間の関係を形式的に記述する。Stanford Encyclopedia of Philosophy (SEP) は、哲学の百科事典として広く利用されている。
これらのリソースは Phil Ecosystem にとって重要なデータソースであるが、以下の点で Phil Ecosystem のアプローチは異なる：(1) PhilPapers と InPhO は主に英語圏の分析哲学に焦点を当てており、非西洋哲学のカバレッジが限定的である。Phil Ecosystem は多言語・多伝統を対象とする。(2) これらのリソースは「索引」としての機能を持つが、「分析」のためのツールは提供しない。Phil Ecosystem は索引（phil.concepts.db）と分析ツール（philmap, philtext）を統合する。(3) 解釈の多重性への対応は、いずれのリソースにもない Phil Ecosystem 固有の機能である。

11.4 Bioconductor / BioPython との比較

Bioconductor[10] は、R ベースのゲノム解析エコシステムであり、Phil Ecosystem の最も重要な設計参照である（第 3 章で詳述）。BioPython[26] は、Python ベースのバイオインフォマティクスライブラリ群であり、Phil Ecosystem の Python 側の設計に影響を与えている。
Phil Ecosystem がこれらから学んだ主要な設計パターンは：(1) 共通データコンテナ（SE → PhilCorpus）による相互運用性、(2) アノテーションデータベース（org.Hs.eg.db → phil.concepts.db）の分離、(3) データリポジトリ（ExperimentHub → PhilCorpusHub）の標準化である。
Phil Ecosystem が独自に追加した要素は：(1) 解釈の多重性への対応（ConceptSpans の interpretation_id）、(2) 非古典論理サポート、(3) R/Python 二言語戦略（Bioconductor は R のみ）である。

11.5 多言語埋め込みモデル

sentence-transformers[14] は、文レベルの埋め込みを効率的に生成するフレームワークであり、Phil Ecosystem の主要な計算バックエンドである。E5[13] は instruction-following embedding モデルであり、philmap-e5 のベースモデルとして使用されている。
哲学ドメインへの応用における課題と対処については第 7 章および第 10 章で議論した。

12 ロードマップと今後の課題

12.1 実装ロードマップ

Phil Ecosystem の開発は 5 つのフェーズで計画されている。表 12 に、各フェーズの概要を示す。

表 12: 実装ロードマップ

フェーズ	時期	主要タスク	成果物
Phase 0	–2026/04	モノレポ構築、既存コード移行、Python philcore 移行、R philcore 構築、philengine 分離、JJDH 論文投稿	リポジトリ骨格、型定義基盤
Phase 1	2026/05–07	philapi (FastAPI) 実装、R philengine (API client)、R philmap 基本機能、phil-hub 基盤、多バックエンド対応	R↔Python 接続、概念検索 MVP
Phase 2	2026/08–10	philworkbench 動詞体系、phil_dashboard() Shiny、phil.concepts.db 構築、ユーザーテスト、科研費萌芽申請	研究者向け UI
Phase 3	2026/11–27/03	philtext 論証抽出、影響検出、philgraph データインジェスタ、Neo4j バックエンド、解釈追跡	専門分析機能

フェーズ	時期	主要タスク	成果物
Phase 4	2027/04–	phil-texts パッケージ群、PyPI/CRAN 公開、ドキュメント整備、コミュニティ構築、国際学会発表	外部公開版

12.2 コミュニティ形成の課題

Bioconductor の成功は、技術基盤だけでなく、コミュニティの形成に大きく依存している。年 2 回のカンファレンス、活発なメーリングリスト、パッケージレビューシステムが、品質の高いエコシステムを支えている。
Phil Ecosystem においても、以下のコミュニティ形成が課題となる：

1. **哲学者とインフォマティクス研究者の対話**: 両者が対等に参加できるワークショップの開催
2. **アノテーションデータの共同構築**: 概念ペアの類似度評価、学派分類の ground truth 構築を共同で行う
3. **パッケージ品質基準**: Bioconductor のパッケージガイドラインに倣った品質基準の策定
4. **倫理的配慮**: 非西洋哲学を西洋的フレームワークで「分析」することの政治性について、継続的な議論が必要

12.3 評価基準の共同構築

Phil Ecosystem の成果を評価する基準は、自然科学の基準をそのまま適用できない。例えば、概念整列の「精度」は、ground truth 自体が論争的であるため、単純な精度・再現率で評価することは困難である。
以下の評価基準を提案する：

1. **発見的有用性**: 計算的探索が、研究者が自力では到達できなかった概念的つながりを示唆したか
2. **解釈的一貫性**: 計算的分析の結果が、当該伝統の専門家の直観と整合するか
3. **研究実践への統合度**: 研究者が日常的にツールを使用しているか（usage analytics）
4. **学術的成果への貢献**: Phil Ecosystem の利用が査読付き論文の出版に寄与したか

13 まとめ

本論文では、哲学研究のための計算基盤エコシステム Phil Ecosystem の設計と実装を報告した。

13.1 本質的な貢献

Phil Ecosystem の本質的な貢献は、以下の 3 点に集約される。
第一に、**哲学データの標準化**である。PhilCorpus データコンテナは、Bioconductor の Summarized-

Experiment に倣い、哲学テキスト・埋め込み・特徴量・概念アノテーションを統合的に格納する標準データ構造を提供する。加えて、解釈の多重性という哲学固有の次元を第一級にサポートする点で、SummarizedExperiment を超える設計を実現した。
第二に、**計算的手法の日常化**である。約 15 の動詞（phil_read, phil_explore, phil_compare, phil_plot 等）によるファサードインターフェースにより、7 層・13 パッケージの複雑なエコシステムを研究者にとって透明にした。哲学ドメイン特化のファインチューニング済み埋め込みモデル（philmap-e5-v2, Spearman $\rho = 0.976$）により、言語・伝統を超えた概念検索を実用的な精度で実現した。
第三に、**非西洋哲学の第一級サポート**である。四句分別、場所論理、矛盾許容論理、弁証法論理の非古典論理を西洋の古典論理と同格に扱い、40 以上の哲学学派（西洋 18、東アジア 8、南アジア 9、イスラーム 4）の分類体系を提供する。これにより、西洋的カテゴリへの還元なしに、各伝統固有のフレームワークでの分析が可能となった。

13.2 「哲学のパラダイムシフト」への展望

Phil Ecosystem は、「計算が哲学に対して、望遠鏡が天文学に対して果たした役割を果たす」というビジョンを掲げた。現時点では、このビジョンは部分的にしか実現されていない。しかし、和辻の「間柄」の探索例（第 4 章）が示したように、計算的探索は個人の認知限界を超えた概念的つながりの発見をすでに可能にしている。
望遠鏡がガリレオに木星の衛星を「見せた」とき、それは既存のプトレマイオス的宇宙観に対する直接的な反証となった。Phil Ecosystem が哲学者に「見せる」ものは、既存の哲学的理論の反証ではなく、個人の読解では到達できなかった概念的親縁関係の候補である。その候補の哲学的意義を評価するのは依然として人間の仕事であるが、候補の発見自体が不可能であった状態からの脱却は、哲学研究の可能性の空間を拡張するものである。
最終的に Phil Ecosystem が目指すのは、哲学者が日常的に計算ツールを使い、「この概念は他の伝統ではどう理解されているか」「この論証は他の文脈でも有効か」「この解釈は他の解釈とどう関係するか」といった問いに対して、計算なしでは到達できなかった回答を、低い障壁で得られる世界である。そのような世界において、哲学は依然として「思考の技芸」であり続けるが、その思考が到達できる範囲は、いかなる個人の読解能力をも超えたものとなるだろう。

参考文献

[1]和辻哲郎.『倫理学』上巻. 岩波書店, 1937.

[2]M. Heidegger. *Sein und Zeit.* Max Niemeyer Verlag, 1927.（邦訳：『存在と時間』細谷貞雄訳, ちくま学芸文庫, 1994.）

[3]西田幾多郎.『善の研究』. 弘道館, 1911.

[4]J. L. Garfield (trans.). *The Fundamental Wisdom of the Middle Way: Nāgārjuna's Mūla-madhyamakakārikā.* Oxford University Press, 1995.

[5]T. Metz. Toward an African moral theory. *The Journal of Political Philosophy*, 15(3):321–341, 2007.

[6]M. Buber. *Ich und Du.* Insel Verlag, 1923.（邦訳：『我と汝・対話』植田重雄訳, 岩波文庫, 1979.）

[7]E. Levinas. *Totalité et Infini: Essai sur l'extériorité*. Martinus Nijhoff, 1961. (Eng. trans.: *Totality and Infinity*, Duquesne University Press, 1969.)

[8]H. L. Dreyfus. *Being-in-the-World: A Commentary on Heidegger's Being and Time, Division I*. MIT Press, 1991.

[9]W. J. Richardson. *Heidegger: Through Phenomenology to Thought*. Martinus Nijhoff, 1963.

[10]W. Huber, V. J. Carey, R. Gentleman, et al. Orchestrating high-throughput genomic analysis with Bioconductor. *Nature Methods*, 12(2):115–121, 2015.

[11]M. Morgan, V. Obenchain, J. Hester, H. Pagès. SummarizedExperiment: SummarizedExperiment container. R package version 1.14.0, Bioconductor, 2018.

[12]S. F. Altschul, W. Gish, W. Miller, E. W. Myers, D. J. Lipman. Basic local alignment search tool. *Journal of Molecular Biology*, 215(3):403–410, 1990.

[13]L. Wang, N. Yang, X. Huang, et al. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*, 2022.

[14]N. Reimers and I. Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-Networks. In *Proceedings of EMNLP-IJCNLP 2019*, pp. 3982–3992, 2019.

[15]P. He, J. Gao, W. Chen. DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing. *arXiv preprint arXiv:2111.09543*, 2021.

[16]D. Bourget and D. J. Chalmers. What do philosophers believe? *Philosophical Studies*, 170(3):465–500, 2014.

[17]M. Niepert, C. Buckner, C. Allen. A dynamic ontology for a dynamic reference work. In *Proceedings of the 8th ACM/IEEE-CS Joint Conference on Digital Libraries*, pp. 288–297, 2008.

[18]F. Moretti. *Distant Reading*. Verso Books, 2013.

[19]A. Burdick, J. Drucker, P. Lunenfeld, T. Presner, J. Schnapp. *Digital_Humanities*. MIT Press, 2012.

[20]D. M. Blei, A. Y. Ng, M. I. Jordan. Latent Dirichlet Allocation. *Journal of Machine Learning Research*, 3:993–1022, 2003.

[21]P. Thagard. *Conceptual Revolutions*. Princeton University Press, 1992.

[22]H. Leitgeb. *The Stability of Belief: How Rational Belief Coheres with Probability*. Oxford University Press, 2017.

[23]T. Williamson. Model-building in philosophy. In R. Blackford and D. Broderick (eds.), *Philosophy's Future: The Problem of Philosophical Progress*, pp. 159–172. Wiley, 2017.

[24]E. Gamma, R. Helm, R. Johnson, J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1995.

[25]T. S. Kuhn. *The Structure of Scientific Revolutions*. University of Chicago Press, 1962.（邦訳：『科学革命の構造』中山茂訳, みすず書房, 1971.）

[26]P. J. A. Cock, T. Antao, J. T. Chang, et al. Biopython: freely available Python tools for computational molecular biology and bioinformatics. *Bioinformatics*, 25(11):1422–1423, 2009.

[27]CAS. SciFinder: A research discovery tool. Chemical Abstracts Service, American Chemical Society. https://scifinder.cas.org/

[28]和辻哲郎.『風土 —— 人間学的考察』. 岩波書店, 1935.

[29]龍樹（ナーガールジュナ）.『中論（Mūlamadhyamakakārikā)』.（邦訳：『中論 —— 縁起・空・中の思想』三枝充悳訳, レグルス文庫, 1984.）

[30]孔子.『論語』.（金谷治訳注, 岩波文庫, 1999.）

[31]西田幾多郎. 場所.『哲学研究』, 第 12 巻第 1 号, 1926.（『西田幾多郎全集』第 4 巻所収, 岩波書店.）

[32]大橋良介.『日本的なもの、ヨーロッパ的なもの』. 新潮選書, 2001.

[33]J. D. McAuliffe, L. P. Findeis, B. M. Brady. Digital humanities and Islamic & Middle Eastern studies. *International Journal of Middle East Studies*, 51(1):133–136, 2019.

[34]S. Ramsay. *Reading Machines: Toward an Algorithmic Criticism*. University of Illinois Press, 2011.

[35]D. M. Berry (ed.). *Understanding Digital Humanities*. Palgrave Macmillan, 2012.